TP : Déploiement Django sur Render

avec une base de données PostgreSQL

Django 4.2+ | Python 3.10+ | Render Free Tier

Objectifs

- Préparer une application Django pour la production
- Utiliser PostgreSQL comme base de données (obligatoire)
- Servir les fichiers statiques avec WhiteNoise
- Déployer automatiquement via Git sur Render
- Configurer les variables d'environnement et les scripts de build

1. Prérequis

Avant de commencer, assurez-vous d'avoir :

- Un projet Django fonctionnel en local (créez-en un avec django-admin startproject monprojet)
- Un compte sur Render (render.com)
- Git installé et votre projet poussé sur GitHub ou GitLab
- Python 3.10+ et pip à jour

2. Installation des dépendances de production

Dans votre environnement virtuel, installez les packages nécessaires :

```
pip install gunicorn whitenoise dj-database-url psycopg2-binary python-dotenv
```

Package	Rôle
gunicorn	Serveur WSGI pour Django en production
whitenoise	Sert les fichiers statiques (CSS, JS, images)
dj-database-url ≥ 2.0	Configure la base de données depuis une variable d'environnement
psycopg2-binary	Pilote PostgreSQL pour Django
python-dotenv	Charge les variables depuis un fichier .env en développement local

Mettez à jour votre fichier requirements.txt :

```
pip freeze > requirements.txt
```

3. Configuration de Django pour la production

3.1. Fichiers statiques avec WhiteNoise

Modifiez le fichier settings.py :

```
# settings.py

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',  # juste après SecurityMiddleware
    # ... autres middlewares
]

STATIC_URL = 'static/'
STATIC_ROOT = 'staticfiles'

# Configuration du stockage des fichiers statiques (Django 4.2+)
STORAGES = {
    "default": {
```

```
        "BACKEND": "django.core.files.storage.FileSystemStorage",
    },
    "staticfiles": {
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}
```

3.2. Configuration de PostgreSQL (obligatoire en production)

Dans settings.py, configurez la base de données via la variable d'environnement DATABASE_URL :

```
# settings.py
import dj_database_url
import os
from dotenv import load_dotenv

load_dotenv()  # charge le fichier .env en développement local

DATABASES = {
    'default': dj_database_url.config(
        default='sqlite:///db.sqlite3',  # fallback uniquement pour le
développement local
        conn_max_age=600,
    )
}
```

> **En production sur Render**
> La variable DATABASE_URL pointera toujours vers une instance PostgreSQL fournie par Render.
> Render intègre automatiquement sslmode=require dans l'URL de connexion.
> Aucune exécution en production n'utilisera SQLite.

3.3. Sécurité : ALLOWED_HOSTS et CSRF_TRUSTED_ORIGINS

Ajoutez l'URL de votre service Render dans settings.py. Remplacez monapp-django par le nom réel de votre service :

```
# settings.py

ALLOWED_HOSTS = [
    'monapp-django.onrender.com',
    '127.0.0.1',
    'localhost',
]

# Obligatoire pour Django 4.x : autorise les requêtes POST (admin, formulaires)
CSRF_TRUSTED_ORIGINS = [
    'https://monapp-django.onrender.com',
]
```

> ⚠ **CSRF_TRUSTED_ORIGINS est obligatoire avec Django 4.x**
> Sans ce paramètre, toutes les requêtes POST (connexion admin, formulaires) échouent avec une erreur 403 Forbidden.
> Pensez à l'ajouter avant de tester votre application déployée.

3.4. Désactiver DEBUG en production

```
# settings.py
DEBUG = os.environ.get('DEBUG', 'False') == 'True'

# Valeur par défaut pour le développement local uniquement
SECRET_KEY = os.environ.get('SECRET_KEY', 'dev-only-insecure-key-change-me')
```

3.5. Fichier .env pour le développement local

Créez un fichier .env à la racine du projet pour stocker vos variables locales :

```
# .env — NE PAS committer dans Git
DEBUG=True
SECRET_KEY=votre-cle-secrete-locale-longue-et-aleatoire
DATABASE_URL=sqlite:///db.sqlite3
```

Ajoutez ces entrées dans votre fichier .gitignore :

```
# .gitignore
.env
staticfiles/
__pycache__/
*.pyc
db.sqlite3
```

4. Scripts de déploiement

À la racine de votre projet (là où se trouve manage.py), créez les deux scripts suivants.

4.1. Script de build : build.sh

```
#!/usr/bin/env bash
# build.sh — exécuté par Render lors de la construction

set -o errexit

# Installer les dépendances
pip install -r requirements.txt

# Collecter les fichiers statiques
python manage.py collectstatic --no-input

# Appliquer les migrations (base PostgreSQL fournie par DATABASE_URL)
python manage.py migrate
```

Rendez-le exécutable :

```
chmod +x build.sh
```

4.2. Script de démarrage : run.sh

```
#!/usr/bin/env bash
# run.sh — commande de démarrage du service Web

set -o errexit

# Render injecte le port d'écoute via la variable $PORT
# Remplacez 'monprojet' par le nom réel de votre projet Django
gunicorn monprojet.wsgi:application --bind 0.0.0.0:$PORT --workers 2
```

Rendez-le exécutable :

```
chmod +x run.sh
```

> ⚠ **--bind 0.0.0.0:$PORT est obligatoire sur Render**
> Render injecte dynamiquement le port d'écoute via la variable $PORT.
> Sans cette option, Gunicorn écoute sur le port 8000 et l'application n'est pas accessible.
> Le déploiement échoue alors avec l'erreur : 'No open ports detected'.

5. Déploiement sur Render

5.1. Créer une base de données PostgreSQL sur Render

1. Connectez-vous à votre dashboard Render (dashboard.render.com)
2. Cliquez sur New + → PostgreSQL
3. Remplissez les champs suivants :

Champ	Valeur recommandée
Name	ma-base-django (ou le nom de votre choix)
Database	laisser par défaut
User	laisser par défaut
Plan	Free (suffisant pour le TP)

4. Cliquez sur Create Database
5. Une fois la base créée, accédez à son tableau de bord et copiez la valeur Internal Database URL. Gardez-la précieusement.

> ℹ **Internal vs External Database URL**

Utilisez toujours l'Internal Database URL pour le service Web (communication interne sur le réseau Render, plus rapide).
L'External Database URL sert uniquement pour les connexions depuis votre machine locale (psql, DBeaver, TablePlus...).

5.2. Créer le service Web

6. Cliquez sur New + → Web Service
7. Connectez votre dépôt GitHub (ou GitLab) et sélectionnez votre projet Django
8. Configurez les champs suivants :

Champ	Valeur
Name	monapp-django (URL : monapp-django.onrender.com)
Root Directory	laisser vide si requirements.txt est à la racine
Environment	Python 3
Build Command	bash build.sh
Start Command	bash run.sh
Instance Type	Free

5.3. Variables d'environnement (essentielles)

Dans la même page de création, ajoutez les variables d'environnement suivantes :

Variable	Valeur
DATABASE_URL	Collez l'Internal Database URL copiée à l'étape 5.1
DEBUG	False
SECRET_KEY	Chaîne longue et aléatoire (voir commande ci-dessous)
PYTHON_VERSION	3.12.0 (ou la version que vous utilisez)

Pour générer une SECRET_KEY sécurisée :

```
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

⚠ **Ne jamais coder en dur SECRET_KEY ou DATABASE_URL dans votre code**
Ces valeurs doivent toujours provenir des variables d'environnement.
Ne les ajoutez jamais dans settings.py ou dans tout fichier commité dans Git.

9. Cliquez sur Create Web Service

Render va alors :

- Cloner votre dépôt Git
- Exécuter bash build.sh (installation des dépendances, collectstatic, migrations)
- Lancer bash run.sh (démarrage de Gunicorn sur $PORT)

5.4. Alternative : déploiement as-code avec render.yaml

Vous pouvez éviter la configuration manuelle sur l'interface en ajoutant un fichier render.yaml à la racine de votre projet. Render le détecte automatiquement lors de la connexion du dépôt.

```
# render.yaml
services:
  - type: web
    name: monapp-django
    env: python
    buildCommand: bash build.sh
    startCommand: bash run.sh
    envVars:
      - key: DEBUG
        value: 'False'
      - key: SECRET_KEY
        generateValue: true   # Render génère automatiquement une valeur sécurisée
      - key: DATABASE_URL
        fromDatabase:
          name: ma-base-django
          property: connectionString

databases:
  - name: ma-base-django
    databaseName: monapp
    plan: free
```

6. Vérification du bon déploiement

Une fois le déploiement terminé (quelques minutes) :

10. Cliquez sur l'URL fournie par Render, par exemple https://monapp-django.onrender.com
11. Vous devriez voir votre application Django s'afficher correctement
12. Vérifiez que les fichiers statiques (CSS, images) sont bien chargés via l'inspecteur du navigateur
13. Visitez /admin pour confirmer que l'interface d'administration fonctionne
14. Pour vous assurer que PostgreSQL est bien utilisé, connectez-vous à la base via psql ou un client graphique (DBeaver, TablePlus) avec l'External Database URL et vérifiez la présence des tables Django (auth_user, django_migrations, etc.)

7. Dépannage fréquent

Problème	Solution

L'application plante au démarrage	Consultez les logs sur Render (onglet Logs). L'erreur est visible en clair.
ALLOWED_HOSTS error	Ajoutez 'monapp-django.onrender.com' dans ALLOWED_HOSTS dans settings.py
Erreur 403 Forbidden sur l'admin ou un formulaire	Ajoutez CSRF_TRUSTED_ORIGINS = ['https://monapp-django.onrender.com'] dans settings.py
Fichiers statiques absents (CSS manquant)	Vérifiez que WhiteNoise est dans MIDDLEWARE et que STORAGES est défini correctement
Erreur de connexion PostgreSQL	Vérifiez que DATABASE_URL est bien définie dans les variables Render. Utilisez l'Internal URL.
No open ports detected	Vérifiez que run.sh contient bien --bind 0.0.0.0:$PORT dans la commande gunicorn
psycopg2 non trouvé	Vérifiez que psycopg2-binary figure dans requirements.txt
collectstatic échoue sur Render	Vérifiez que SECRET_KEY est définie dans les variables d'environnement Render

8. Bonnes pratiques

- Ne jamais utiliser * dans ALLOWED_HOSTS en production : spécifiez le nom exact de votre service Render
- Gérer toutes les variables sensibles (SECRET_KEY, DATABASE_URL) via les Environment Variables de Render, jamais dans le code source
- Redéploiement automatique : par défaut, Render rebuild votre application à chaque git push sur la branche principale
- Pour les fichiers uploadés par les utilisateurs : utilisez un stockage externe (Cloudinary, AWS S3) car Render ne persiste pas les fichiers sur le disque du service Web entre les déploiements
- Gérer les environnements : utilisez python-dotenv avec un fichier .env en local, et les variables Render en production
- Surveiller les logs : l'onglet Logs de Render est votre premier réflexe en cas de problème

9. Rendu attendu

À la fin du TP, l'étudiant doit être capable de :

- Fournir l'URL publique de son application Django fonctionnant sur Render
- Montrer que la base de données PostgreSQL est utilisée (par exemple via un modèle créé et visible dans l'admin)
- Expliquer le rôle de gunicorn, whitenoise, dj-database-url et psycopg2
- Expliquer pourquoi CSRF_TRUSTED_ORIGINS est nécessaire avec Django 4.x
- Expliquer pourquoi --bind 0.0.0.0:$PORT est indispensable dans run.sh

Félicitations !

Vous avez déployé une application Django robuste avec PostgreSQL sur Render, prête pour la production.